

Anexo Python básico - Programando en biología

Software a usar

- Python (Google Colab)

Recursos Online

- [Google Colab - Guía de inicio](#)
- [Python Tutorial \(Documentación oficial\)](#)
- [Pandas - User Guide](#)
- [Pandas Cheat Sheet](#)
- [Curso Kaggle Learn - Python](#)
- [Curso Kaggle Learn - Pandas](#)

Introduccion a Python

Python es actualmente uno de los lenguajes más utilizados en bioinformática, ciencia de datos e inteligencia artificial. Su popularidad se debe a su sintaxis clara y sencilla, así como a la gran cantidad de bibliotecas especializadas que ofrece para el procesamiento, análisis y visualización de datos.

Programando en Python

Ahora que tenemos una idea de la interfaz de **Google Colab** vamos a ver como se crean los programas.

Antes de empezar, es útil entender que un programa en **Python** no es más que una secuencia de instrucciones que la computadora ejecuta.

Estas instrucciones pueden ser de distintos tipos. Por ejemplo:

- Crear o modificar **variables**, que almacenan información.
- Llamar a **funciones**, que realizan una tarea específica.
- Escribir estructuras como **if**, **for** o **while**, que permiten tomar decisiones o repetir acciones (las veremos más adelante).

Una **función** es un bloque de código que realiza una tarea específica. Algunas funciones ya vienen incluidas en **Python**, otras son proporcionadas por librerías y también podemos crear nuestras propias funciones. Más adelante veremos esto con mayor detalle.

Las funciones pueden recibir uno o más **argumentos**, es decir, la información que necesitan para realizar su tarea. Por ejemplo:

```
print("Hola Mundo!")
```

En este caso, `print()` es la función y `"Hola Mundo!"` es el argumento que le estamos pasando.

También existen funciones que reciben varios argumentos. Por ejemplo:

```
print("Hola", "Mundo")
```

En este caso la función recibe dos argumentos y los imprime separados por un espacio.

Ayuda de las funciones

En **Google Colab** pueden obtener información sobre una función simplemente dejando el cursor sobre su nombre. También pueden escribir el nombre de la función seguido de `?`, por ejemplo:

```
print?
```

Esto mostrará una breve descripción de la función y los argumentos que acepta.

 **5)** En la primera **celda de código** escriban lo siguiente y ejecútenla presionando Shift + Enter o haciendo clic en el botón ►.

```
print("Hello World!")
```

`print` es equivalente al `echo` de **bash** y cuando lo usamos decimos que *imprimimos* a la variable. Puede ser que usemos las frases "*imprimir por terminal*", "*imprimir por consola*", o "*imprimir por pantalla*" de forma intercambiable.

Como muestra el código anterior, los argumentos de una función en **Python** se escriben entre paréntesis y, si hay más de uno, se separan con comas.

Al ejecutar la celda deberían ver:

```
Hello World!
```

 **6)** Modifiquen ahora la misma celda para que imprima:

```
print("Hola Bioinformática!")
```

Vuelvan a ejecutarla usando Shift + Enter.

Cada vez que modifiquen una celda deberán volver a ejecutarla para que los cambios tengan efecto.

 **7)** Si desean conservar los cambios realizados en el notebook, guárdenlo en su cuenta de Google Drive mediante **Archivo** :material-arrow-right: **Guardar una copia en Drive**.

Por defecto, Google Colab guardará el notebook en la carpeta **Mi unidad** :material-arrow-right: **Colab Notebooks** de su Google Drive. El archivo tendrá la extensión **.ipynb**.

A diferencia de un *script* tradicional de **Python** (archivos con extensión **.py**), un notebook de **Google Colab** guarda no solo el código, sino también el texto explicativo, las figuras, las tablas y los resultados obtenidos durante la ejecución de cada celda.

Python: Variables

Como dijimos cuando vimos **Bash**, diferentes lenguajes tienen elementos similares, pero se usan ligeramente diferente. Vamos a entonces a aprender como se usan las variables en **Python**.

 **1)** Escriban en una celda de código la siguiente línea y ejecútenla:

```
bacteria = "Escherichia coli"
```

En **Python**, las variables se asignan utilizando el símbolo **=**. Una vez creada una variable, podemos utilizarla simplemente escribiendo su nombre.

¿Se acuerdan del panel **Variables** que vimos al comienzo del TP? Si lo abren ahora, deberían ver la variable **bacteria** junto con su valor.

 **2)** En una nueva celda escriban:

```
print(bacteria)
```

La función **print()** muestra el valor de una variable por pantalla.

Al ejecutar la celda deberían obtener:

```
Escherichia coli
```

Al igual que en otros lenguajes, las variables permanecen almacenadas en memoria y pueden reutilizarse en las celdas siguientes sin necesidad de volver a definir las. Esto es una gran ventaja, ya que nos permite trabajar sin tener que ejecutar todo el notebook desde el principio cada vez que hacemos un cambio.

Sin embargo, un gran poder conlleva una gran responsabilidad. Como las variables permanecen en memoria, es posible estar utilizando un valor "viejo" si modificamos el código pero no volvemos a ejecutar la celda correspondiente. Esto puede generar resultados inesperados y hacer más difícil encontrar errores.

Por esta razón, una buena práctica es ejecutar las celdas del notebook en orden, de arriba hacia abajo, para asegurarse de que todas las variables tengan el valor esperado.

También es buena práctica comentar las instrucciones más importantes para que el código pueda ser entendido y reutilizado por otras personas. Es algo así como el **cuaderno de laboratorio bioinformático**.

En un notebook también es posible mostrar el contenido de una variable escribiendo únicamente su nombre:

```
bacteria
```

En este caso, **Google Colab** mostrará automáticamente el contenido de la variable porque es la **última instrucción de la celda**. Si hubiera más líneas de código después de `bacteria`, ese valor ya no se mostraría automáticamente.

Ambas formas permiten ver el contenido de la variable. Sin embargo, cuando escribimos programas es recomendable utilizar `print()` para dejar explícito que queremos mostrar ese resultado.

 **3)** Escriban y ejecuten las siguientes celdas:

```
bacteria
```

```
bacteria  
  
print("Fin")
```

¿Qué diferencia observan entre ambas celdas? ¿Por qué creen que ocurre?

Comentarios

Los comentarios en **Python** se hacen de la misma forma que en **Bash**, ignorando todo lo que está después de un `#`.

```
saludo = "Hola Mundo!"  
# saludo = "Esto no va a hacer nada"  
  
print(saludo) # print("Esto tampoco")
```

Los comentarios son muy útiles para explicar qué hace una parte del código o dejar notas para uno mismo o para otras personas que vayan a reutilizar el programa.

Es posible comentar o descomentar varias líneas de código al mismo tiempo. Para ello, seleccionen las líneas que desean comentar y presionen `Ctrl + /`

Números

En **Python** existen dos tipos principales de variables numéricas: los números enteros (**int**) y los números reales (**float**).

 **4)** Ejecuten el siguiente código celda por celda y observen cómo se va modificando el valor de la variable **n**.

```
n = 2 # n es una variable de tipo int
print(n)
```

```
# Podemos realizar diferentes operaciones matemáticas
n = ((n + 2) * 2) ** 2
print(n)
```

```
n = ((n - 2) / 2) ** (1/2)
print(n)
```

Observen que el valor final de **n** ya no es un número entero sino un número real.

Si no saben de qué tipo es una variable, pueden utilizar la función `type()`, la cual devuelve el tipo de la variable. Por ejemplo:

```
print(type(n))
```

Cadenas de caracteres o *Strings*

En **Python** las cadenas de caracteres, comúnmente llamadas *strings*, tienen el tipo de variable **str**. En Python, las cadenas de caracteres (strings) se representan mediante texto encerrado entre comillas. Pueden utilizarse tanto comillas simples ('...') como comillas dobles ("..."), y ambas son equivalentes. La elección suele depender del estilo o de si la cadena contiene comillas del otro tipo.

 **5)** Ejecuten las siguientes celdas, lean los comentarios y observen cómo se va modificando el valor de la variable **frase**.

- Código

```
frase = "Había"
print(frase)
```

```
frase = frase + " una"
print(frase)
```

```
frase = frase + " vez"
print(frase)
```

```
frase = frase + "..."  
print(frase)
```

- Código con comentarios

```
frase = "Había"    # frase es una variable de tipo str  
  
# Podemos concatenar strings utilizando el operador +  
frase = frase + " una"  
  
# Agregamos otra palabra, incluyendo un espacio al comienzo  
frase = frase + " vez"  
  
# Agregamos tres puntos, sin espacio entre "vez" y los puntos  
frase = frase + "..."  
  
print(frase)
```

6) Averigüen de qué tipo es la variable `frase`.

En **Python**, muchas operaciones sobre los *strings* se realizan mediante **métodos**, es decir, funciones asociadas al propio objeto. Los métodos se invocan escribiendo un punto (.) después del nombre de la variable.

Es importante notar que **cada tipo de objeto tiene sus propios métodos**. Algunos métodos son exclusivos de un determinado tipo de objeto, mientras que otros pueden existir en varios tipos pero realizar tareas diferentes. Por ejemplo, los *strings* tienen métodos como `upper()` o `replace()`, mientras que las listas tienen métodos como `append()`, que veremos más adelante.

Por ejemplo, podemos convertir un texto a mayúsculas.

```
frase.upper()
```

7) Ejecuten las siguientes líneas y observen qué ocurre:

```
print(frase)  
  
frase.upper()  
  
print(frase)
```

Noten que el método `upper()` **no modifica** la variable `frase`, sino que devuelve un nuevo *string* con el resultado. Es importante notar que no todos los métodos se comportan de la misma manera. Algunos, como `upper()`, devuelven un nuevo objeto sin modificar el original, mientras que otros sí modifican el objeto sobre el que se aplican.

Si queremos conservar el resultado de `upper()`, podemos asignarlo a una nueva variable:

```
frase_mayuscula = frase.upper()

print(frase_mayuscula)
```

O bien reemplazar el contenido de la variable original:

```
frase = frase.upper()

print(frase)
```

Otros métodos nos permiten reemplazar parte del texto o contar cuántas veces aparece un carácter:

```
frase.replace("Había", "Existe")
```

```
frase.count("a")
```

Se pueden hacer muchas más cosas con *strings*, como extraer partes del texto (*substrings*), buscar si una palabra aparece dentro de otro texto o dividir un *string* en varias partes. Iremos aprendiendo estas herramientas a medida que las necesitemos.

Ejercicio 1 - Variables simples

En este ejercicio vamos a practicar el uso de variables numéricas y *strings* en **Python**.

1. Creen dos variables numéricas, asígnenles un valor e impriman el resultado de realizar su suma. Luego hagan lo mismo pero con resta y multiplicación.
2. Asignen el valor 0 a una nueva variable e intenten dividir uno de los números anteriores por ella.
 - ¿Qué ocurre? ¿Cómo informa Python este error?
3. Creen una variable de tipo *string* con la siguiente secuencia de ADN:

```
ATGGCTAACCGTTACGAATGCTAA
```

- Impriman la longitud de la secuencia utilizando la función `len()`.
- Cuenten cuántas veces aparece la base "A" utilizando el método `count()`.
- Generen la secuencia de ARN reemplazando todas las T por U utilizando el método `replace()`, y guardenla como una nueva variable.
- ¿Cuántos aminoácidos codificaría esta secuencia?

Listas

En **Python**, una de las estructuras de datos más utilizadas es la **lista** (*list*). Una lista es una colección ordenada de elementos, que se escriben entre corchetes (`[]`) y cuyos elementos se separan con comas.

Las listas pueden contener elementos de distintos tipos, aunque en bioinformática es habitual trabajar con listas cuyos elementos son todos del mismo tipo.

```
# Lista con strings
lista_frase = ["Había", "una", "vez..."]
```

Como acceder a los elementos de una lista

Como pueden ver, es posible acceder a un elemento individual de la lista utilizando corchetes.

Índices

En **Python**, el primer elemento de una lista tiene índice **0**. Por ejemplo, `lista_frase[0]` devuelve "Había", mientras que `lista_frase[1]` devuelve "una".

Si ya programaron en otros lenguajes, tengan presente que no todos utilizan la misma convención para numerar los elementos de una colección.

 **1)** Ejecuten las siguientes celdas:

```
print(lista_frase)
```

```
print(lista_frase[0])
```

```
print(lista_frase[1])
```

¿A qué elemento de la lista corresponde cada índice?

Ahora probemos con una lista de números:

```
lista_numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

 **2)** Ejecuten las siguientes celdas:

```
print(lista_numeros[5])
```

¿El índice coincide con el valor almacenado? Noten que el **índice** no representa el valor almacenado, sino la **posición** del elemento dentro de la lista. El índice 5 corresponde al **sexto elemento** de la lista.

```
# También podemos crear listas de números consecutivos utilizando la función
`range()`.
lista_rango = list(range(0,12))
print(lista_rango)
```

 **3)** Respondan las siguientes preguntas:

- ¿El número 12 está incluido en la lista?
- Utilicen `range()` para generar una lista igual a `lista_numeros`.

Además de acceder a un único elemento, también es posible acceder a una parte de una lista utilizando la sintaxis `inicio:fin` (Slices).

 **4)** Ejecuten las siguientes celdas:

```
print(lista_numeros[2:5])
```

```
print(lista_numeros[:4])
```

¿Qué elementos devuelve cada caso?

Al igual que con `range()`, el índice final **no está incluido**.

Por ejemplo, `lista_numeros[2:5]` devuelve los elementos ubicados en las posiciones **2, 3 y 4**.

También es posible combinar listas utilizando el operador `+`:

```
lista_hueco = list(range(1,4)) + list(range(7,11))
print(lista_hueco)
```

De la misma forma, también podemos agregar elementos al final de una lista utilizando `+`:

```
print(lista_numeros + [11, 12])
```

Noten que la lista original `lista_numeros` no se modifica. Si quieren conservar el resultado, deberán asignarlo a una variable.

Una forma de **modificar directamente la lista original** es utilizando el método `append()`, que agrega un elemento al final de la lista:

 5) Ejecuten las siguientes celdas:

```
print(lista_numeros)
```

```
lista_numeros.append(11)

print(lista_numeros)
```

A diferencia del operador `+`, el método `append()` **sí modifica** la lista original.

Las listas de **Python** pueden contener elementos de distintos tipos:

```
lista_mixta = [1, 2, "asd"]

print(type(lista_mixta))

print(type(lista_mixta[0]))
print(type(lista_mixta[2]))
```

Cada elemento conserva su tipo original. Sin embargo, si intentamos realizar operaciones incompatibles obtendremos un error. Por ejemplo, no tiene sentido sumar un número con un *string*.

```
print(lista_mixta[0] + lista_mixta[1]) # válido

print(lista_mixta[0] + lista_mixta[2]) # error

print(lista_mixta[2] + 3)               # error
```

Diccionarios

Otra estructura de datos muy utilizada en **Python** es el **diccionario** (*dictionary*). Un diccionario almacena pares **clave-valor** (*key-value*), donde cada clave identifica un valor asociado. Cada **clave** funciona como el nombre de un dato y cada **valor** es la información asociada a esa clave.

Se crean utilizando llaves (`{}`) y cada par clave-valor se escribe de la forma **clave: valor**:

```
gen = {
    "nombre": "TP53",
    "esencial": True,
    "expresion": 5234
}
```

A diferencia de las listas, donde accedemos a los elementos por su **posición** (índice), en los diccionarios accedemos a los valores utilizando su **clave**. Podemos acceder a un valor indicando su clave entre corchetes:

```
print(gen["nombre"])

print(gen["expresion"])
```

```
TP53
5234
```

Los diccionarios son muy útiles para representar información de un único objeto. Por ejemplo, los datos de un gen, una proteína o un paciente, donde cada característica tiene un nombre asociado.

Python: Estructuras lógicas

Condicionales y Booleanos

Al igual que en **Bash**, **Python** también tiene condicionales **if**, aunque la sintaxis es ligeramente diferente.

- Código

```
numero = 42

print(numero, "es un número")

if numero > 10:
    print(numero, "es mayor a 10")
else:
    print(numero, "es menor o igual a 10")
```

- Código con comentarios

```
numero = 42

print(numero, "es un número")

# if es la estructura más utilizada para realizar condicionales.
# Después del if va la condición y termina con dos puntos (:)
if numero > 10:
    # Todo el código indentado (más a la derecha) debajo del if se ejecuta
    # únicamente si la condición es verdadera.
    # Si la condición es falsa, este bloque se saltea.
    #
    # En Python la indentación es obligatoria y define qué
    # instrucciones pertenecen al if. Google Colab la agrega
    # automáticamente al presionar Enter después de los dos puntos (:).
```

```

print(numero, "es mayor a 10")
else:
    # Todo el código indentado debajo del else se ejecuta
    # únicamente cuando la condición del if es falsa.
    print(numero, "es menor o igual a 10")

```

A diferencia de otros lenguajes, Python no utiliza llaves ({}) para delimitar bloques de código. En su lugar utiliza la indentación, es decir, la cantidad de espacios al comienzo de cada línea. Todas las líneas con la misma indentación pertenecen al mismo bloque de código. Google Colab agrega automáticamente la indentación cuando corresponde, por ejemplo al presionar ++enter++ luego de escribir un if, for, while o una función.

La condición del if es en realidad una variable de tipo booleano, es decir, una variable que solo puede tomar uno de dos valores: True (verdadero) o False (falso).

Hay tres formas principales de obtener variables booleanas:

```

# Asignando True o False directamente
booleano1 = True

# Utilizando un comparador
numero1 = 5
booleano2 = numero1 > 10

# Combinando booleanos mediante operadores lógicos
booleano3 = booleano1 and booleano2

```

Como mostramos en la tercera forma de generar variables booleanas, se pueden hacer operaciones entre booleanos usando la llamada **álgebra booleana**. Esto es un tema muy amplio, pero al momento de programar nos interesarán principalmente tres operaciones: **AND**, **OR** y **NOT**.

El **AND** y el **OR** son operaciones entre dos booleanos, mientras que **NOT** es una operación que se aplica a un único booleano.

- El **AND** es el "Y", devolviendo **True** { .green_text } solo cuando ambos booleanos son **True** { .green_text }. En **Python** se escribe con **and**.
- El **OR** es el "O", devolviendo **True** { .green_text } cuando al menos uno de los dos booleanos es **True** { .green_text }. En **Python** se escribe con **or**.
- El **NOT** es el "NO", invirtiendo el valor del booleano (es decir, devuelve **True** { .green_text } solo si el booleano era **False** { .red_text }). En **Python** se escribe con **not**.

► Detalles de ****AND****, ****OR**** y ****NOT****

Booleano1	Booleano2	AND (Booleano1 and Booleano2)	OR (Booleano1 or Booleano2)
 True	 True	 True	 True
 True	 False	 False	 True

Booleano1	Booleano2	AND (Booleano1 and Booleano2)	OR (Booleano1 or Booleano2)
 False	 True	 False	 True
 False	 False	 False	 False

Booleano1	NOT (not)
 True	 False
 False	 True

En **Python**, una variable booleana ya representa una condición, por lo que no es necesario compararla explícitamente contra **True** o **False**.

Veán el siguiente ejemplo:

- Código

```
llueve = True
tengo_paraguas = True

if llueve and not tengo_paraguas:
    print("Me mojo")
else:
    print("No me mojo")
```

- Código con comentarios

```
llueve = True
tengo_paraguas = True

# llueve es equivalente a escribir llueve == True
# not tengo_paraguas es equivalente a escribir tengo_paraguas == False
if llueve and not tengo_paraguas:
    # Este bloque solo se ejecuta si ambas condiciones son True:
    #   - llueve
    #   - not tengo_paraguas
    print("Me mojo")
else:
    # Si una o ambas condiciones son False (como ocurre en este ejemplo,
    # donde not tengo_paraguas es False), se ejecuta este bloque.
    print("No me mojo")
```

► Comparar dos valores

Para comparar dos valores en Python (por ejemplo, en un condicional) utilizamos operadores de comparación. Estos devuelven un booleano: **True** o **False**.

Para comparar si dos valores son iguales utilizamos el operador **==**

`==` significa "es igual a".

No utilizamos `=` porque este operador se utiliza para asignar un valor a una variable.

Ejemplo:

```
numero = 42

if numero == 42:
    print("El número es 42")
```

Los operadores de comparación son:

Operador	Significado
<code>==</code>	igual a
<code>!=</code>	distinto de
<code>></code>	mayor que
<code><</code>	menor que
<code>>=</code>	mayor o igual que
<code><=</code>	menor o igual que

► Paréntesis en las condiciones

En este ejemplo no fue necesario utilizar paréntesis. Sin embargo, cuando las condiciones son más complejas y contienen comparaciones (`<`, `>`, `==`, etc.), es recomendable utilizarlos para que el código sea más fácil de leer y evitar errores.

Por ejemplo:

```
edad = 25
tiene_credencial = True

if (edad >= 18) and tiene_credencial:
    print("Puede ingresar")
```

Ciclos

Ciclo *For*

En **Python**, los ciclos `for` permiten recorrer los elementos de una colección (como una lista) o una secuencia de números. Se escriben de la siguiente forma:

- Código

```
for i in range(1, 11):  
    print(i)
```

- Código con comentarios

```
# for es una de las estructuras más usadas para hacer ciclos.  
  
# i es el nombre de la variable que va cambiando en cada iteración.  
# Podríamos llamarla de cualquier otra forma, por ejemplo numero,  
# pero por convención suele llamarse i.  
  
# range(1, 11) genera una secuencia con los números del 1 al 10.  
# El segundo número (11) no está incluido.  
for i in range(1, 11):  
    # Todo el código indentado se ejecuta una vez por cada valor de i.  
    print(i)
```

En este caso, el `for` recorre una secuencia de números del **1** al **10** e imprime el valor de `i` en cada iteración.

También podemos recorrer directamente los elementos de una lista previamente declarada:

- Código

```
lista_colores = ["rojo", "amarillo", "verde"]  
  
for color in lista_colores:  
    print(color)
```

- Código con comentarios

```
lista_colores = ["rojo", "amarillo", "verde"]  
  
for color in lista_colores:  
    # color es la variable que va cambiando en cada iteración del for  
    # En este caso va tomando los valores de los diferentes elementos  
    # de la lista lista_colores.  
    print(color)
```

► Ciclo While

El ciclo `for` es muy útil, pero tiene la limitación de que generalmente conocemos de antemano cuántas iteraciones va a realizar. Sin embargo, esto no siempre es posible. Para esos casos existe el ciclo `while`, que repite un bloque de código mientras se cumpla una determinada condición.

- Código

```
```python
contador = 1

while contador <= 10:
 print(contador)

 contador = contador + 1
```
```

- Código con comentarios

```
```python
Declaramos una variable contador con el valor 1
contador = 1

Todo el código indentado debajo del while se ejecutará
mientras la condición contador <= 10 sea True.
while contador <= 10:
 # Imprimimos el valor de contador
 print(contador)

 # Aumentamos el valor de contador en una unidad
 contador = contador + 1
```
```

Tal vez no se den cuenta, pero este programa hace exactamente lo mismo que el **for** del ejemplo anterior. La variable **contador** comienza valiendo **1** y al finalizar cada iteración aumenta en una unidad. Cuando **contador** llega a **11**, la condición **contador <= 10** deja de cumplirse, el ciclo termina y el número **11** nunca llega a imprimirse.

En este caso no tendría mucho sentido utilizar un ciclo **while**, ya que el mismo problema puede resolverse fácilmente con un **for**. Sin embargo, el **while** resulta muy útil cuando no sabemos de antemano cuántas iteraciones serán necesarias.

Por ejemplo, si queremos imprimir las potencias de 2 menores que **1000**:

- Código

```
```python
numero = 1

while numero < 1000:
 print(numero)

 numero = numero * 2
```
```

- Código con comentarios

```
```python
Declaramos una variable numero con el valor 1 (2º)
numero = 1

El ciclo continuará mientras numero sea menor que 1000
while numero < 1000:
 # Imprimimos el valor actual
 print(numero)

 # Multiplicamos numero por 2 para obtener
 # la siguiente potencia de 2
 # 2¹ = 2
 # 2² = 4
 # 2³ = 8
 # 2⁴ = 16
 # etc.
 numero = numero * 2
```
```

Si bien también existen formas de resolver este último ejemplo utilizando un **for**, el objetivo es mostrar que un ciclo **while** permite repetir una acción una cantidad **indeterminada** de veces, dependiendo del resultado de una condición.

► Loops infinitos

Al utilizar un ciclo `while` o `for` hay que tener cuidado de no escribir un programa que genere un loop infinito, es decir, un ciclo cuya condición nunca deje de ser True.

Si esto ocurre, el programa continuará ejecutándose indefinidamente hasta que lo interrumamos manualmente.

Ejercicio 2 - Estructuras lógicas

El contenido GC es la proporción de bases **G** y **C** presentes en una secuencia de ADN.

Creen una variable llamada **secuencia** que contenga la siguiente secuencia:

```
ATGCGATCGGATTAACGCGCTT
```

Luego escriban un programa que:

- Cree una variable llamada **gc** con valor 0.
- Recorra la secuencia utilizando un ciclo **for**.
- Si la base es **"G"** o **"C"**, aumente **gc** en 1.
- Imprima la cantidad total de bases **G** o **C** presentes en la secuencia.

► Tip

Van a necesitar combinar un ciclo `for` con una estructura `if` utilizando el operador lógico `or`.

¿Recuerdan los métodos que vimos para trabajar con *strings*? ¿Se les ocurre una forma de obtener la cantidad de bases **G** y **C** sin utilizar un ciclo `for`?

Python: Funciones

Como ya mencionamos cuando hablamos de los ciclos, es común en programación querer realizar una tarea muchas veces con datos ligeramente diferentes. Para evitar repetir código existen las **funciones**, que además de formar parte de **Python** y de las distintas librerías, también pueden ser creadas por nosotros.

Supongamos que queremos calcular la temperatura de melting de un primer. Podemos definir una función de la siguiente forma:

```
def temperatura_melting(primer):
    A = primer.count("A")
    T = primer.count("T")
    G = primer.count("G")
    C = primer.count("C")

    tm = 2 * (A + T) + 4 * (G + C)

    return tm

primer1 = "ATGCGTACGATCG"
primer2 = "GCGTACGCGGATCGCTAGCTACGATG"

tm1 = temperatura_melting(primer1)
tm2 = temperatura_melting(primer2)

print(tm1)
print(tm2)
```

En este ejemplo:

- `def` indica que estamos definiendo una función.
- `temperatura_melting` es el nombre de la función.
- `primer` es el parámetro de entrada y contiene la secuencia del primer.
- `return` indica cuál será el valor que devolverá la función.

Las funciones permiten reutilizar código y evitar escribir varias veces las mismas instrucciones. Imaginen que el cálculo ocupara 20 o 30 líneas de código: con una función solo habría que escribirlo una vez.

También es posible definir funciones con más de un parámetro. Incluso algunos parámetros pueden tener un **valor por defecto**, que será utilizado si no indicamos otro valor.

```
def peso_proteina(sequencia, peso_promedio_aminoacido=110):
    peso = len(sequencia) * peso_promedio_aminoacido
```

```
    return peso

proteina1 = "MKTLLVAGCGTALAVASAPVAAEAKKAGVEAKKAAEAGVEAAGVEAKKAGVEVVV"

peso1 = peso_proteina(proteina1)
peso2 = peso_proteina(proteina1, peso_promedio_aminoacido=115)

print(peso1)
print(peso2)
```

Como pueden ver, si no especificamos el parámetro `peso_promedio_aminoacido`, Python utiliza automáticamente su valor por defecto (110 Da). Sin embargo, podemos modificarlo cuando llamamos a la función. Esto permite utilizar la misma función con diferentes valores sin necesidad de modificar el código que se encuentra dentro de ella.

Consola de Python

- Comando `help()`